

Civic.ai RAG Progress Report

Current implementation status for the Bangla-English government-service RAG thesis prototype.

1. Project Goal

Civic.ai is being developed as a citizen-facing government service assistant for Bangladesh. The implementation follows the broad NextRAG idea, but adapts it from financial PDFs to government-service guidance, starting with passport FAQ data.

2. Current Dataset

The active dataset is a manually curated passport FAQ JSON file. Raw PDFs are preserved as source artifacts, but the RAG index is built from structured FAQ records because they are better for retrieval, metadata, evaluation, and citation.

Artifact	Path / Description
Processed FAQ	data/processed/passport_rag_clean.json
Raw PDFs	data/raw/government_services/Passport.pdf and brta_user_manual.pdf
Generated chunks	data/interim/passport_faq_chunks.jsonl
Vector index	data/processed/chroma_passport_bge_m3

3. Implemented Pipeline

The current pipeline is: processed FAQ JSON -> metadata-rich chunks -> BGE-M3 dense embeddings -> ChromaDB -> BM25 -> weighted RRF fusion -> reranking -> local Ollama generation -> chatbot UI.

Layer	Current Implementation
Ingestion	Loads structured FAQ records from JSON.
Chunking	One metadata-rich chunk per FAQ record.
Dense retrieval	BAAI/bge-m3 embeddings stored in ChromaDB.
Sparse retrieval	BM25 over the same chunk text.
Fusion	Weighted RRF, currently dense=0.6 and BM25=0.4.
Reranking	Cross-encoder hook implemented; lexical fallback active if model is not cached.
Generation	Local Ollama models through LangChain.
Deployment	Local browser chatbot at http://127.0.0.1:7860.

4. Local LLMs

The chatbot is not tied to one LLM. It can compare multiple local generators while using the same retrieved evidence.

Model	Role
llama3.2	Fast local baseline.
llama3	Stronger LLaMA-family baseline; prompt was adjusted to improve source use.
qwen2.5:7b	Alternative model-family baseline; early sample performed strongly.

5. Implemented Files

File	Purpose
app.py	Local web chatbot and /chat API.
src/pipeline.py	Shared RAG pipeline wrapper.
src/retrieval/hybrid_retriever.py	Dense + BM25 retrieval and RRF fusion.
src/reranking/reranker.py	Cross-encoder reranker with lexical fallback.
src/generation/ollama_generator.py	Grounded prompt and local LLM generation.
scripts/build_index.py	Builds ChromaDB index from processed FAQ JSON.
scripts/query_rag.py	Command-line query script.
scripts/compare_models.py	Runs the same evidence through multiple LLMs.
scripts/evaluate_retrieval.py	Numeric retrieval evaluation.
scripts/evaluate_generation.py	Numeric generated-answer evaluation.

6. Current Numeric Evaluation

Retrieval smoke-test results using the indexed FAQ questions:

Method	N	Recall@1	Recall@5	MRR	nDCG@5
CivicRAG	64	1.000	1.000	1.000	1.000
Simple RAG	64	1.000	1.000	1.000	1.000

Generated-answer sample results:

Model	N	Semantic Similarity	Token F1	Source Acc.	Citation Acc.
llama3.2	3	0.854	0.618	1.000	1.000
qwen2.5:7b	3	0.951	0.919	1.000	1.000

Important limitation: the current retrieval benchmark is a smoke test because it uses the same FAQ questions that are indexed. For thesis-quality results, the next dataset should include paraphrased English, Bangla, and code-mixed questions with expected source IDs.

7. Next Steps

Priority	Next Work
1	Curate a stronger JSON/JSONL dataset with service, category, language, source, keywords, and Bangla fields.
2	Create a separate evaluation set with paraphrased English, Bangla, and code-mixed questions.
3	Download/cache BGE reranker or evaluate a lighter reranker alternative.
4	Run full Simple RAG vs CivicRAG comparisons for llama3.2, llama3, and qwen2.5:7b.
5	Add citation/faithfulness review and final thesis tables.